

Software Requirements Specification (SRS)

Project: AuraNet - Autonomous, Self-Healing Network Security Framework

Prepared by: Diaa Hanna

Document Context: Chapter 4 - Analytical Study & System Specifications

Overview: This document provides a precise analytical study of the AuraNet system, aiming to define all use cases, system specifications, and the engineering mechanisms adopted to achieve every functional and non-functional requirement.

1. Analysis Structure & Methodology

AuraNet is designed as a security framework and underlying infrastructure upon which developers can build their applications. Its nature differs fundamentally from traditional software projects, requiring a paradigm shift in the standard software engineering analysis approach.

1.1 The Paradigm Shift

Because AuraNet operates at the lower layers of network infrastructure and functions autonomously without human intervention, traditional Human-Computer Interaction models (like standard Use Case Diagrams) are less relevant. Forcing these models would misrepresent the system's true engineering reality.

Consequently, "Use Cases" are strictly limited to administrative operations and system configuration. Additionally, because AuraNet is primarily a security system, the analysis incorporates **Threat Modeling** and **Misuse Diagrams** to comprehensively reflect the project's core objectives.

1.2 The Alternative Methodology

The analysis follows this specific structural methodology:

- **Developer to Underlying Infrastructure Interaction:** Use cases limited to administration and configuration.
- **Misuse Diagrams:** Profiling malicious behaviors and potential system exploitation.
- **Application-Level Threat Modeling:** Analyzing attacks targeting hosted workloads that the system aims to block.
- **Infrastructure-Level Threat Modeling:** Assessing risks targeting the AuraNet components themselves.

2. Detailed Requirements (Mini-SRS)

2.1 Functional Requirements (FR)

AuraNet must strictly comply with Zero Trust architecture based on the NIST SP 800-207 standard. This core objective is broken down into the following functional requirements:

Req ID	Description
FR-1	The system must fully secure and encrypt all communications regardless of their network location.
FR-2	The system must never grant implicit trust to any device simply based on its presence within the network infrastructure.
FR-3	The system must enforce the "Least Privilege" principle, granting only the access necessary to complete a network task.
FR-4	The system must evaluate access requests using dynamic policies that consider user identity, application state, device characteristics, and contextual behaviors.
FR-5	The system must conduct continuous monitoring to evaluate the security posture of connected devices and immediately revoke access upon detecting anomalies or vulnerabilities.
FR-6	The system must strictly enforce Authentication and Authorization before allowing any connection.
FR-7	The system must continuously collect and analyze telemetry regarding network state and connections to optimize and update security policies.
FR-8	The system must provide mechanisms to adapt and learn the behavior of new, previously unknown network components (solving the static rules problem).
FR-9	The system must process network data locally at the edge nodes to avoid centralizing massive raw data logs (Big Data Storage problem).

FR-10	The system must distribute security analysis and model training workloads in parallel across edge nodes to reduce response time and central resource consumption.
FR-11	The system must minimize network bandwidth consumption by transmitting only analytical summaries (e.g., Model Weights) to the center, rather than raw data streams.
FR-12	The system must protect network flow privacy and comply with data protection standards (e.g., GDPR) by not centralizing sensitive data.
FR-13	The system must automatically trigger self-healing mechanisms as a reactive response to remediate errors and vulnerabilities, securely closing the security loop.
FR-14	The system must provide a centralized Dashboard enabling administrators to monitor the live security cycle, track node statuses, and review event logs.

2.2 Non-Functional Requirements (NFR)

Req ID	Attribute	Description
NFR-1	Critical Recovery Time	The system must achieve a critical response time (from attack occurrence to completed auto-healing/isolation) of no more than 5 seconds .
NFR-2	Detection Time	The system must detect security threats within a maximum window of 2 seconds .

NFR-3	Non-blocking Performance	The system must not create a network bottleneck. Packets must pass transparently without any noticeable latency overhead.
NFR-4	Cloud-Native Support	The system must preserve native Kubernetes characteristics, primarily scalability and transparent resource management.
NFR-5	Manageability	The system must offer flexible management via GUI and CLI, allowing modification of vital parameters (e.g., learning rates, toggling federated learning).
NFR-6	Standard Security	The system must effectively thwart standard cyber threats, particularly those listed in the OWASP Top 10.
NFR-7	Model Convergence	The Federated Learning model must achieve mathematical convergence and complete an aggregation round within 15 minutes .
NFR-8	AI Model Security	The system must incorporate robust defenses against AI-specific attacks, such as Data Poisoning or malicious weight manipulation.
NFR-9	Topological Flexibility	The system must integrate seamlessly into various complex network topologies regardless of node count or distribution.
NFR-10	Technology Constraint	The infrastructure must utilize eBPF technology at the OS kernel level to accelerate security operations and bypass traditional network performance bottlenecks.

3. System Actors

3.1 Human Actors

- **Workload Developer (Cluster/Infrastructure Admin):** The primary user responsible for installing, configuring, and managing AuraNet on the Kubernetes cluster.
- **Application User:** Individuals interacting with internal dashboards or applications, generating the East-West traffic monitored by the system.

3.2 Threat Actors

- **Application Attacker:** Aims to exploit software vulnerabilities in deployed workloads to breach containers.
- **Infrastructure Attacker:** Targets the core infrastructure components (Kubernetes or AuraNet itself) to take over the cluster.
- **Insider Threat:** Malicious actions originating from within the network via abused legitimate privileges or compromised internal components.

3.3 Autonomous System Entities

- **AuraNet Autonomous System:** The primary software actor operating independently to manage the entire security loop (monitoring, federated learning analysis, auto-healing decisions). *Note: Kubernetes and Cilium fall under the umbrella of this actor.*

4. Use Cases & Misuse Cases

4.1 Standard Use Cases

- **Lifecycle & Environment:**
 - UC-01 System Installation

- UC-02 Application Deployment
- UC-03 System Uninstallation
- UC-04 Application Removal
- UC-05 Kubernetes Management
- **Policy Management & Configuration:**
 - UC-06 Configuration Adjustment
 - UC-07 Network Policy Definition
 - UC-08 Threat Matrix Configuration
 - UC-09 Manual Identity Assignment
 - UC-10 Exception Management (Temporary suspensions for maintenance)
- **Monitoring & Operations:**
 - UC-11 System Monitoring (Live Dashboard)
 - UC-12 Auto-healing Testing
 - UC-13 Traffic Generation

4.2 Misuse Cases

- MUC-01 **Application Vulnerability Exploitation:** Attempting to exploit workload flaws to access data.
- MUC-02 **System Component Exploitation:** Targeting underlying Kubernetes/AuraNet infrastructure to disrupt operations.
- MUC-03 **Lateral Network Scanning:** Post-breach horizontal reconnaissance to discover and exploit adjacent cluster services.

5. Threat Modeling

5.1 Security Assumptions

- SA-1 **Trusted Kubernetes Control Plane:** Core components (API Server, etcd) are uncompromised.
- SA-2 **Trusted OS Kernel:** The Linux kernel is pristine, and the eBPF Verifier operates correctly.
- SA-3 **Cryptographic Soundness:** mTLS and WireGuard algorithms remain mathematically secure against current capabilities.
- SA-4 **Physical Security:** Physical access to servers is secure; out of project scope.
- SA-5 **Infrastructure Admin Trustworthiness:** The admin possessing root access is fully trusted.
- SA-6 **Supply Chain Integrity:** K8s/AuraNet container images are pulled from untainted sources.
- SA-7 **Trusted Cilium CNI:** The network agent handling lower-level routing and WireGuard encryption is uncompromised.

5.2 Application-Level Threats

- **Network Threats:** Malicious packets targeting specific application vulnerabilities.
- **Runtime Threats:** Unauthorized execution behaviors inside a container (e.g., blocked Syscalls).
- **Lateral Movement:** Horizontal network traversal by an attacker holding an initial foothold.

5.3 Out-of-Scope Threats

Certain threats are deliberately excluded due to architectural logic and project scope:

1. **DDoS Attacks:** Excluded due to the availability of standard tools (e.g., Fail2ban), the unsuitability of AI for instant DDoS mitigation, and architectural constraints. East-West traffic scope means large-scale botnet flooding is structurally impossible, and Zero Trust authentication prevents arbitrary packet flooding.

2. **Application-Level Encryption:** If developers encrypt payloads within the application code, eBPF sensors cannot inspect the traffic. This violates the cloud-native separation of concerns. A *Workload Developer Guide* will explicitly state that relying on application-layer encryption voids AuraNet's analytical coverage.
3. **Non-HTTP Protocols (Demo Phase):** Implementing microsegmentation requires deep protocol understanding. The initial iteration includes protocol drivers strictly for HTTP and PostgreSQL.
4. **Runtime DDoS (Resource Exhaustion):** Excluded because native Linux kernel features (e.g., cgroups) already handle process-spawning limits efficiently.

6. System Specifications (Implementation & Design)

6.1 Functional System Specifications (SS-FR)

- SS-FR-1 Implement kernel-level encryption utilizing WireGuard via Cilium.
- SS-FR-2 Enforce a "Default Deny/Drop" posture for all new network resources.
- SS-FR-3 Use eBPF-backed microsegmentation to restrict ingress/egress to explicitly allowed ports, protocols, and identities, dropping invalid packets in the kernel.
- SS-FR-4 The Central Controller computes a Dynamic Trust Score based on ML outputs and identity data.
- SS-FR-5 Inject deep eBPF probes to capture runtime telemetry and syscalls, dynamically lowering the trust score upon anomaly detection.
- SS-FR-6 Cryptographic Identities of communicating nodes are verified in the datapath before TCP/UDP establishment.
- SS-FR-7 Data pipelines route network logs to local models to continuously update applied policies.
- SS-FR-8 Integrate Continuous Delivery for Machine Learning (CD4ML) principles inherent in the federated architecture.

- SS-FR-9 Extract features locally on Worker Nodes and immediately discard raw data.
- SS-FR-10 Execute training epochs locally to distribute CPU/Memory load.
- SS-FR-11 Transmit only mathematical Model Gradients/Weights to the Aggregator.
- SS-FR-12 Anonymize payloads and restrict PII from leaving the edge node.
- SS-FR-13 Deploy virtual patching protocols to restore trust to compromised components autonomously.
- SS-FR-14 Serve telemetry through standardized APIs to a Web GUI for live metric rendering.

6.2 Non-Functional System Specifications (SS-NFR)

- SS-NFR-1 The event lifecycle (alert to actual kernel blocking) is optimized to remain under 5.0 seconds under standard load.
- SS-NFR-2 The local Inference Engine classifies features and alerts within 2.0 seconds.
- SS-NFR-3 Utilize a "Shadow Stream" mechanism, cloning packets asynchronously for analysis to guarantee non-blocking primary traffic flow.
- SS-NFR-4 Ensure programmatic compatibility with K8s virtual resource managers.
- SS-NFR-5 Provide a secure RESTful API acting as middleware for CLI/GUI hot-reloading without cluster reboots.
- SS-NFR-6 Map monitoring signatures to known standard injection/privilege-escalation patterns.
- SS-NFR-7 Tune communication rounds so the global Aggregation Round completes reliably under 15 minutes.
- SS-NFR-8 The Aggregator incorporates robust Outlier Detection algorithms to drop anomalous weights (Poisoning mitigation).
- SS-NFR-9 Validate architecture operability across varying compute node topologies.
- SS-NFR-10 Write all packet interception and syscall tracing strictly in eBPF (C language) for direct, safe kernel execution.

7. Logical System Components

1. **Sensors:**

- *Runtime Monitor:* Captures OS/kernel-level events.
- *Network Monitor:* Captures network traffic data.

2. **Analyzer / Filter:** Evaluates events to identify threats.

- *Runtime Analyzer*
- *Network Analyzer*

3. **Central Controller:** The core brain computing the Dynamic Trust Score for each node.

4. **Remediator (Corrector):** Executes automated corrective actions when a component's trust score drops below the critical threshold.

5. **Identity & Access Manager (IAM):** Manages workload identities and handles authentication/authorization.

6. **Policy Bootstrapper:** Bootstraps the environment by applying base network policies built exclusively on pre-approved communication flows.

7. **Federated Aggregator:** Collects model weights from Local AI Agents to update the global detection model without centralizing raw data.

8. Defense Specifications

8.1 Infrastructure Defense Mechanisms

- SS-SEC-1 **Model Poisoning Defense:** Local AI Agents are trained strictly on benign traffic. The Central Aggregator utilizes robust mathematical filtering to detect and discard anomalous weights.
- SS-SEC-2 **eBPF Evasion Prevention:** Leverages K8s structural isolation. Even if an attacker gains container root access, namespaces strictly prevent them from reaching host-level kernel space to tamper with eBPF probes.

- SS-SEC-3 **System Components Protection:** AuraNet components are isolated in a dedicated, locked-down Namespace. Interaction is restricted to asynchronous, memory-safe backend packet reading.

8.2 Workload Defense Mechanisms

- SS-SEC-4 **Network Threats Mitigation:** The Network Analyzer intercepts and inspects packets in real-time, matching patterns and dropping malicious packets before they reach the application.
- SS-SEC-5 **Runtime Threats Mitigation:** The Runtime Analyzer continually monitors syscalls, instantly blocking unauthorized execution behaviors at the container level.
- SS-SEC-6 **Lateral Movement Prevention:** The Policy Bootstrapper enforces cryptographically-bound microsegmentation policies, blinding the attacker to adjacent nodes and confining them entirely to the compromised container.